

Decision Tree Analysis: Classification, Regression, and Cost Complexity Pruning

Data Science and Machine Learning Module

January 13, 2026

Abstract

This report details the implementation and optimization of Decision Tree algorithms for both classification and regression tasks. We explore the fundamental split logic using Gini Impurity, visualize the recursive partitioning of feature space, and analyze the impact of tree depth on predictive accuracy. A significant portion of the project focuses on Cost Complexity Pruning (CCP) using the hyperparameter α . By sweeping through alpha values and analyzing the trade-off between training and validation accuracy, we identify the optimal pruned tree that maximizes generalization and prevents overfitting.

Contents

1	Introduction	3
2	Theory	3
2.1	Split Criteria: Gini Impurity	3
2.2	Regression: Mean Squared Error (MSE)	3
2.3	Cost Complexity Pruning (α)	3
3	Implementation Methodology	4
4	Results	6
4.1	Root Node Selection Using Gini-Impurity	6
4.2	Performing Regression Using Decision Tress	7
4.3	Pruning Performance	8
5	Discussion	9
6	Conclusion	10

1 Introduction

Decision Trees are among the most intuitive and powerful non-parametric supervised learning algorithms used for both classification and regression. They operate by recursively partitioning the feature space into distinct regions, where each region is assigned a specific class label or a continuous value.

In this project, we utilize three distinct datasets to understand how decision trees are used for classification and regression. The `new_placement_data.csv` allows us to study classification based on CGPA and Projects counts. The `regression_tree.csv` provides a basis for understanding how trees approximate complex continuous functions as piecewise constant "staircases." Finally, the `moons_tree.csv` serves as a benchmark for geometric boundary analysis and pruning.

A primary challenge in Decision Tree construction is the tendency of the model to overfit. An unconstrained tree will continue to split until every training point is perfectly isolated, essentially "memorizing" noise. This report explores how hyperparameter tuning—specifically Cost Complexity Pruning (α)—can be used to find the "sweet spot" where a model is complex enough to capture the signal but simple enough to ignore the noise.

2 Theory

2.1 Split Criteria: Gini Impurity

For classification, the tree selects the best split based on Gini Impurity reduction. The Gini index for a set S is defined as:

$$Gini(S) = 1 - \sum_{i=1}^C p_i^2 \quad (1)$$

The algorithm chooses a feature and a threshold that yields the lowest weighted average Gini impurity of the resulting child nodes.

2.2 Regression: Mean Squared Error (MSE)

For regression, the tree predicts the mean value of the target variable within a leaf. The quality of a split is measured by the reduction in Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2 \quad (2)$$

2.3 Cost Complexity Pruning (α)

Cost Complexity Pruning is used to control tree size. The objective function minimized is:

$$R_\alpha(T) = R(T) + \alpha|T| \quad (3)$$

where $R(T)$ is the total impurity (or error) of the leaves, and $|T|$ is the number of terminal nodes. A higher α penalizes larger trees, forcing the model to become simpler.

3 Implementation Methodology

The project was implemented in Python using the `scikit-learn` library. The methodology followed four distinct tasks:

1. **Manual Calculation:** Implementing the Gini Impurity function from scratch to determine the root node of the placement dataset.

```
1 def GiniImpurity(feature, feature_avg):
2
3     gini_total = np.zeros(2999)
4     min_gini = float('inf')
5
6     # Total samples in the placement dataset
7     total_samples = 3000
8
9     for j in range(2999):
10        # Split the target labels 'm' based on the feature
11        threshold
12        leaf1 = m[feature < feature_avg[j]]
13        leaf2 = m[feature >= feature_avg[j]]
14
15        # Process Leaf 1 (Left branch)
16        n1 = len(leaf1)
17        if n1 > 0:
18            P1Yes = np.sum(leaf1 == 1) / n1
19            P1No = np.sum(leaf1 == 0) / n1
20            gini1 = 1 - (P1Yes**2 + P1No**2)
21        else:
22            gini1 = 0 # Pure node or empty set
23
24        # Process Leaf 2 (Right branch)
25        n2 = len(leaf2)
26        if n2 > 0:
27            P2Yes = np.sum(leaf2 == 1) / n2
28            P2No = np.sum(leaf2 == 0) / n2
29            gini2 = 1 - (P2Yes**2 + P2No**2)
30        else:
31            gini2 = 0
32
33        # Weighted Average Gini for the split
34        gini_total[j] = (n1/total_samples)*gini1 + (n2/
35        total_samples)*gini2
36
37        # Updating min_gini
38        if gini_total[j] < min_gini:
39            min_gini = gini_total[j]
```

Listing 1: Manual Gini Impurity Calculation for Root Node Selection

2. **Regression Visualization:** Training trees with varying `max_depth` (e.g., 2 vs. 10) to visualize the difference between a smooth staircase approximation and a "jagged" overfitted line.

```

1 from sklearn.tree import DecisionTreeRegressor, plot_tree
2
3 # 1. Prepare the Data correctly
4 # Explicitly define features to match the single column 'x'
5 features = ['x']
6 X = df[features]
7 y = df['y']
8
9 # 2. Initialize and Train
10 # We set max_depth=2 to observe a simple 4-leaf piecewise
    approximation
11 reg_tree = DecisionTreeRegressor(max_depth=2, random_state=42)
12 reg_tree.fit(X, y)

```

Listing 2: Regression Tree Training and Structural Constraints

3. **Alpha Sweep:** Iterating through $\alpha \in [0, 0.05]$ to calculate training and validation accuracies.

```

1 # 1. Initialize lists to store scores for plotting
2 train_acc_list = []
3 val_acc_list = []
4 alphas = np.arange(0, 0.055, 0.005)
5
6 # 2. Preparation of features
7 X_train = train_df[['x1', 'x2']]
8 y_train = train_df['y']
9 X_val = val_df[['x1', 'x2']]
10 y_val = val_df['y']
11
12 # 3. Loop through alpha values to find the pruning sweet spot
13 for a in alphas:
14     # Initialize and Fit on training data with specific ccp_alpha
15     reg_tree = DecisionTreeRegressor(ccp_alpha=a, random_state=42)
16     reg_tree.fit(X_train, y_train)
17
18     # Calculate R-squared scores for both sets
19     train_score = reg_tree.score(X_train, y_train)
20     val_score = reg_tree.score(X_val, y_val)
21
22     # Store results for visualization
23     train_acc_list.append(train_score)
24     val_acc_list.append(val_score)

```

Listing 3: Hyperparameter Tuning via Cost Complexity Pruning Sweep

4. **Decision Surface Plotting:** Using a custom `plot_surface` function to visualize how boundaries simplify as α increases.

```

1 def plot_surface(model, ax, title):
2     # Use the training features for range and labels
3     X_vals = X_train.values
4     feature_names = X_train.columns
5
6     # Define coordinate bounds for the meshgrid
7     x_min, x_max = X_vals[:, 0].min() - 0.5, X_vals[:, 0].max() + 0.5
8     y_min, y_max = X_vals[:, 1].min() - 0.5, X_vals[:, 1].max() + 0.5

```

```

9
10     xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
11                           np.arange(y_min, y_max, 0.02))
12
13     # Predict class labels over the grid to visualize regions
14     # grid_data is wrapped in a DataFrame to maintain feature name
15     # consistency
16     grid_data = pd.DataFrame(np.c_[xx.ravel(), yy.ravel()], columns
17                               =feature_names)
18     Z = model.predict(grid_data)
19     Z = Z.reshape(xx.shape)
20
21     # Plotting decision regions (contourf) and training samples (
22     # scatter)
23     ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
24     ax.scatter(X_vals[:, 0], X_vals[:, 1], c=y_train, s=30,
25               cmap='RdBu', edgecolors='white', alpha=0.8)
26
27     ax.set_title(title, fontsize=14, fontweight='bold')
28     ax.set_xlabel(feature_names[0])
29     ax.set_ylabel(feature_names[1])
30
31 # Create side-by-side comparison of model complexity
32 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
33
34 plot_surface(tree_1, ax1, f"Unpruned Tree (Alpha=0)\nOverfitted")
35 plot_surface(tree_2, ax2, f"Pruned Tree (Alpha={opt_alpha:.4f})\
36     nGeneralized")
37
38 plt.tight_layout()
39 plt.show()

```

Listing 4: Visualization of Decision Boundaries: Comparing Overfitted vs. Pruned Trees

4 Results

4.1 Root Node Selection Using Gini-Impurity

- The minimum gini impurity for Projects column= 0.4342140137127068
- The minimum gini impurity for CGPA column= 0.3588954074074074

Since gini impurity for CGPA is less than that for Projects, **root node will be CGPA.**

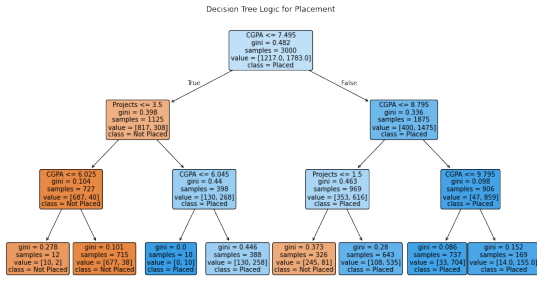


Figure 1: Decision Tree Logic for Placement

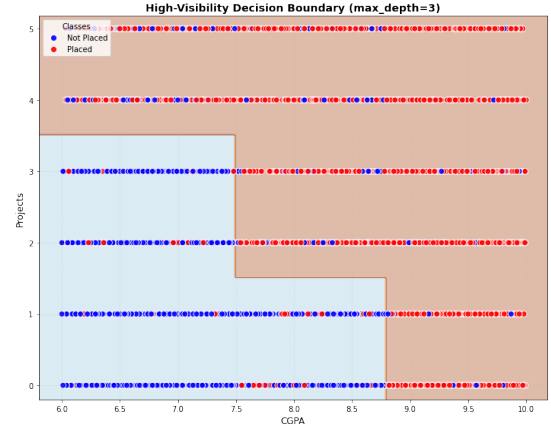


Figure 2: Decision Boundary (max depth=3)

Figure 1 shows the tree with depth equal to 3. Figure 2 represents the decision boundary separating students into placed and not placed classes.

4.2 Performing Regression Using Decision Tress

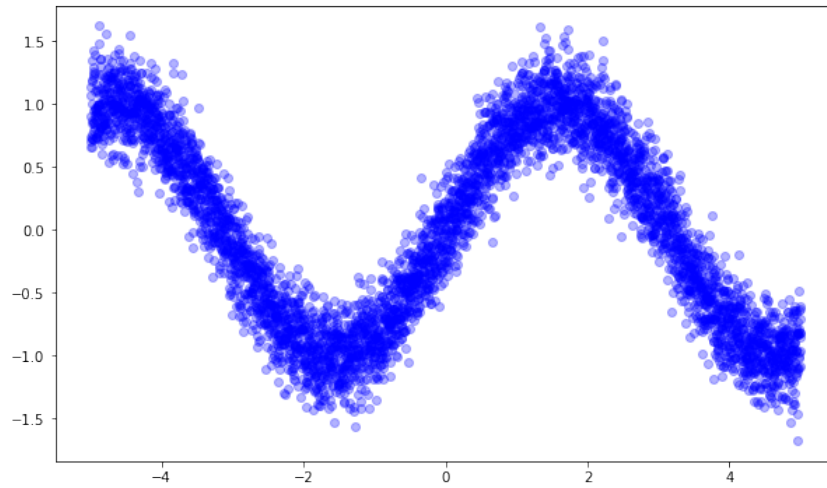


Figure 3: Continuous data Distribution **regression tree.csv**

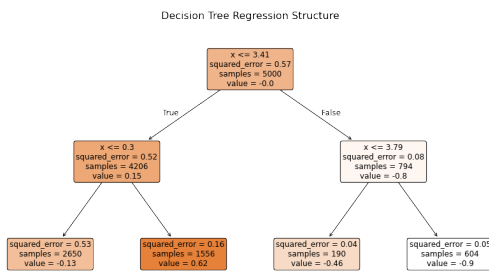


Figure 4: Regression Decision Tree

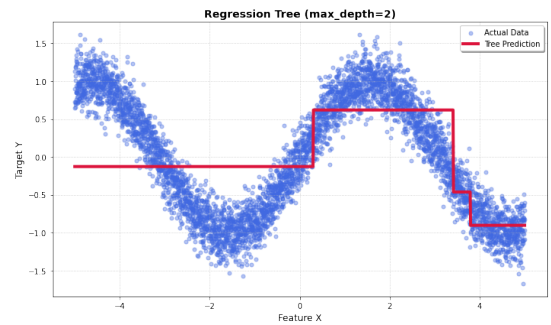


Figure 5: Staircase Decision Boundary (max depth=2)

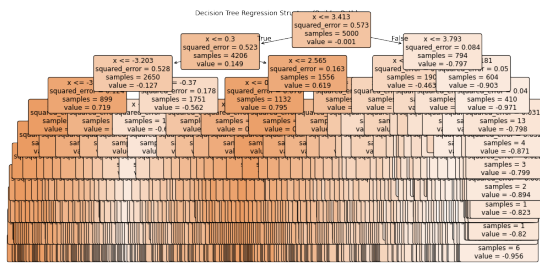


Figure 6: Regression Decision Tree

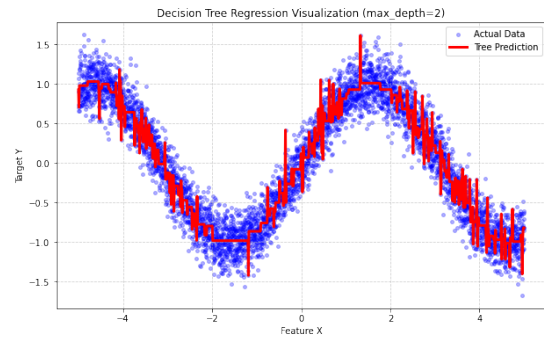


Figure 7: Jagged Decision Boundary (max depth=10)

Unlike classification, where a leaf predicts a "class," in regression, each leaf predicts the **average (mean)** of all training samples falling into that partition. This explains that the resulting model is a "step function."

- For the Shallow tree with depth equals 2 has only four horizontal steps making the model is too simple to capture the local fluctuations of the data. It **"underfits."**
- For the Deep tree (depth = 10) the prediction line is highly 'jagged' and tries to pass through almost every noisy point. The model has "memorized" the noise in the training data. It **"overfits."**

4.3 Pruning Performance

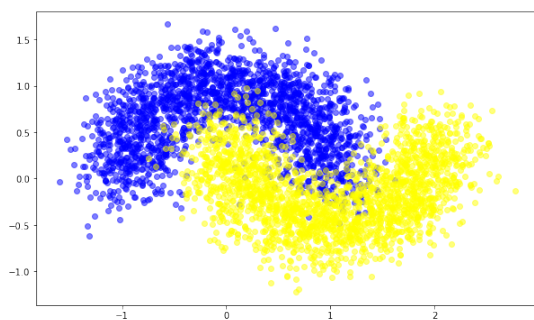


Figure 8: Continuous Data Distribution moons tree.csv

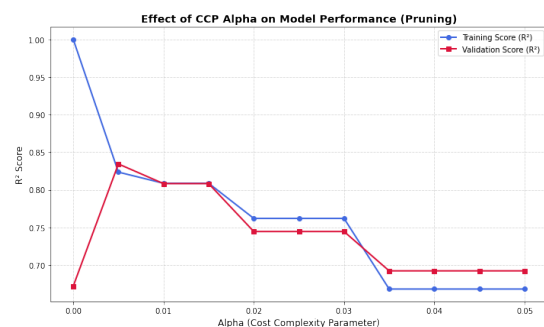


Figure 9: Validation Accuracy and Training Accuracy v/s α

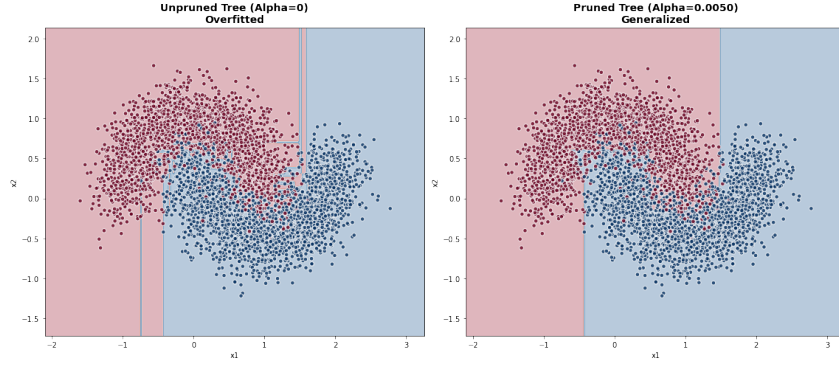


Figure 10: Decision Surfaces for $\alpha = 0$ and $\alpha = 0.005$ respectively

The plot of Training vs. Validation accuracy against α revealed a characteristic crossover at $\alpha = 0.005$. As α increased:

- Training accuracy decreased monotonically as the model lost the ability to memorize specific points.
- Validation accuracy initially increased (as noise-induced branches were removed) before eventually decreasing (as the tree became underfitted).

5 Discussion

The performance and optimization of our models are summarized by the following three pillars:

- **Regression Metric (R^2):** We evaluated performance using the Coefficient of Determination, which represents the proportion of variance in the dependent variable that is predictable from the features.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (4)$$

- **Cost Complexity Equation:** The pruning process was guided by the cost function, where α penalizes the addition of terminal nodes to ensure the model does not "memorize" noise.

$$Cost = \text{Total Impurity Error} + \alpha \times (\text{Number of Terminal Nodes}) \quad (5)$$

- **Geometric Constraint:** Decision boundaries are inherently **axis-parallel** (strictly horizontal or vertical). This occurs because the algorithm splits based on a single feature at a time, resulting in rectangular partitions of the feature space.

Our evaluation relied on the R^2 score to quantify predictive performance, identifying the proportion of variance captured by our regression steps. To optimize this score on unseen data, we applied Cost Complexity Pruning, where the parameter α served as a complexity penalty. This mathematical constraint effectively reduced the number of terminal nodes, transforming overfitted, jagged axis-parallel boundaries into simplified regions that represent the true underlying patterns of the dataset rather than its noise.

6 Conclusion

This project successfully demonstrated the mechanics of Decision Trees. We concluded that while trees are powerful, they are highly sensitive to small variations in data. Pruning via the α parameter is a superior method to manual depth-limiting because it removes nodes that provide the least improvement to the impurity measure relative to their cost. Choosing the optimum pruned tree ensures that the model remains interpretable and performs reliably on unseen real-world data.